Contents

Fix Report: GPU Implementation Plan v1

Date: 2026-05-16 Source review: `gpu_implementation_plan_v1_check.md` Target document: `gpu_implementation_pl`
Fix agent: code-fix skill

Summary

- Findings addressed: 18 of 18 (5 CRITICAL, 9 MODERATE, 4 LOW).
- Files modified: 1 (`gpu_implementation_plan_v1.md`).
- Tests added: 0 (the artifact under fix is a planning document, not source code; test cases are embedded in the plan as acceptance criteria for each phase).
- Test suite: N/A (no executable code in this PR).
- Plan version: bumped to v1.1 (revised header).

Changes Made

CRITICAL (5/5)

R-GPU-001 — Virtual **`StateEvolution::Rate`** on device

- What changed: Phase 2 "Files to Modify" section rewritten. The plan now explicitly forbids tagging the virtual `Rate` method with `MFEM_HOST_DEVICE` and forbids calling `evolution_->Rate(...)` from inside `mfem::forall`. Added a NEW file `friction/state_evolution_kernels.hpp` to the plan, declaring four `MFEM_HOST_DEVICE` free functions (`AgingLawPsiRateDevice`, `SlipLawPsiRateDevice`, `AgingLawRateDevice`, `SlipLawRateDevice`) with explicit signatures matching the host virtuals' `(V, theta_or_psi, Dc[, b, V0, f0])` parameters. Added an `EvolutionKind` enum and a runtime-population step keyed off `evolution_->GetName()` so the device kernel dispatches via a `switch` on the cached enum.
- Location: Phase 2 "Files to Modify" section in gpu_implementation_plan_v1.md.

R-GPU-002 — TPV drivers use raw **`MPI_Init`**, not **`MPIContext`**

- What changed: Phase 0 "Files to Modify" split into two driver groups: (a) BP5 `seas_driver.cpp` (uses `MPIContext`, which calls `Mpi::Init` + `Hypre::Init`; insert `seas::InitDevice` after);
  (b) `tpv102/104/205_driver.cpp` (use raw `MPI_Init`, no `Hypre::Init`; insert `seas::InitDevice` directly after the `MPI_Init` / `MPI_Comm_rank` block at the documented line numbers). Added a worked example for `tpv102_driver.cpp`. Also rewrote the "Initialization order (HARD CONTRACT)" requirement #1 to explicitly describe the two paths and clarify that `Device::Configure` gates `Hypre::InitDevice` on `HYPRE_Initialized()`, making the explicit drivers' "no Hypre" path safe.
- Location: Phase 0 "Files to Modify" and "Detailed Requirements" #1.

R-GPU-003 — **`MAX_DOF = 84`** is wrong for p=4 tet

- What changed: Phase 3 mass-inverse pseudocode now uses `MAX_DOF_TET` instead of `MAX_DOF`, with a comment stating `MAX_DOF_TET = 35` is correct for p=4 tet (formula `(p+1)(p+2)(p+3)/6`). Added an explicit warning that loops MUST be bounded by runtime `ndof`, never by `MAX_DOF_TET`. Memory-budget paragraph recomputed using correct p=4 numbers (15 KiB shape table, ~47 GiB naive G table → factored to 72 MiB Jinv + 47 KiB dshape_ref). Edge-Cases section now aborts at `ndof > 35` with the correct message.
- Location: Phase 3 §"Detailed Requirements" #4–#7, §"Edge Cases".

R-GPU-004 — **`AtomicMax`** is not available in MFEM

- What changed: Phase 2 §"Detailed Requirements" #4 ("V_max reduction") now marks the atomic-max pattern as NOT VIABLE with citation of `general/backends.hpp:94` (only `AtomicAdd<T>` exists) and clarification that CUDA's `atomicMax` has no `double` overload. Only Pattern A (scratch Vector + `Vector::Max()` device-resident reduction) is retained.
- Location: Phase 2 §"Detailed Requirements" #4.

R-GPU-005 — Raw **`atomicAdd`** is not portable

- What changed: (a) Convention constraints at top of plan now require `mfem::AtomicAdd<real_t>` for any accumulation into shared output slots; raw CUDA / HIP `atomicAdd` is forbidden. (b) Phase 4 §"Detailed Requirements" #7 race-resolution Pattern A rewritten to cite `mfem::AtomicAdd<real_t>` from `general/backends.hpp:94` and note the architecture support (≥ CC 6.0 / MI200+).
- Location: §"Convention constraints" (top of plan), Phase 4 #7.

MODERATE (9/9)

R-GPU-006 — **`Device::Configure`** is additive, not single-call

- What changed: §"Edge Cases" of Phase 0 rewritten. Cites `general/device.cpp:242` (`MarkBackend(b) { backends |= b; }`), drops the "abort on conflicting configs" instruction, replaces with a function-static `s_active_cfg + s_initialized` cache pattern that no-ops on identical re-calls and emits a root-rank warning on divergent re-calls. Updated `seas::InitDevice` body in §"Function signature" to include the guard.
- Location: Phase 0 §"Function signature for seas::InitDevice" and §"Edge Cases".

R-GPU-007 — HYPRE iteration count / tolerance claim unrealistic

- What changed: Numerical-constraint bullet for HYPRE PCG+AMG GPU vs CPU loosened from "≤ 1 iteration drift, < 1e−10 L2 diff" to "≤ 2× iteration count, < 1e−6 L2 diff at PCG tol 1e−8" with citation of `linalg/hypre.cpp:1504,1840,…` (`l1Jacobi` vs `l1GS` smoother divergence). Phase 1 acceptance criterion updated to match.
- Location: §"Numerical / performance constraints" and Phase 1 §"Acceptance Criteria".

R-GPU-008 — Volume-kernel pseudocode missing **`.Read()`** / **`.Write()`** captures

- What changed: Phase 3 §"Detailed Requirements" #3 volume kernel pseudocode rewritten to include an explicit HOST SETUP block showing `Q.Read()`, `rhs.ReadWrite()`, `qd.B.Read()`, etc., with `Reshape` views before `mfem::forall`. Added an "Implementer rule" paragraph requiring the same pattern in the mass-inverse and spatial-derivative kernels.
- Location: Phase 3 §"Detailed Requirements" #3.

R-GPU-009 — **`Array<int>`** does not have **`UseDevice(true)`**

- What changed: (a) Phase 4 `FaceTable` struct definition now carries an inline NOTE explaining that `Array<T>` does not have `UseDevice`, and that the implementer must allocate with `Array<T>(n, Device::GetDeviceMemoryType())` and access via `Array::Read()`. Vector members still use `UseDevice(true)`.

(b) Appendix C residency table split into two rows: Vector members (yes UseDevice) and Array members (n/a, device memory type at construction).

- Location: Phase 4 §"Files to Create" / face_table.hpp; Appendix C residency table.

R-GPU-010 — Shared-face ghost GF residency unspecified

- What changed: Phase 4 §"Detailed Requirements" #6 now requires the device shared-face kernel to consume `ghost_gf_full_state_->FaceNbrData()` (vdim=NUM_STATE, R-1601 batched) instead of `ghost_gf_` (vdim=1), to avoid 9 separate ghost exchanges per macro step. Added a bullet that the underlying `ParFiniteElementSpace` must be constructed with `Device::GetDeviceMemoryType()` so the receive buffer lives in device memory. Appendix C gains a `ghost_gf_full_state_` row with the same caveat.
- Location: Phase 4 §"Detailed Requirements" #6; Appendix C.

R-GPU-011 — Non-standard **`"cpu,debug"`** config string

- What changed: Phase 0 §"Files to Modify" `Validate` rule and §"Detailed Requirements" #4 (TOML parsing rule) both narrowed to single-backend set `{"", "cpu", "cuda", "hip", "debug"}`. Multi-backend strings deferred to Phase 7. The `seas::InitDevice` body comment also updated accordingly.
- Location: Phase 0 §"Files to Modify" and §"Detailed Requirements" #4.

R-GPU-012 — Face table ownership ambiguity in Phase 6

- What changed: Phase 6 §"Detailed Requirements" #1 rewritten. Explicitly states `WaveOperator` and `ElasticityDomainOperator` each own their own `FaceTable` instance (no shared instance — they're separate classes with no parent-child relationship). The `FaceTable` struct and the table-builder free function move from `dynamic/kernels/face_table.hpp` to a shared header `common/kernels/face_table.hpp`. Appendix C row for `FaceTable::*` updated to indicate Phase 4 OR Phase 6 owner.
- Location: Phase 6 §"Detailed Requirements" #1; Appendix C.

R-GPU-013 — Curvilinear mesh assumption not flagged

- What changed: Phase 3 §"Edge Cases" now contains an explicit curvilinear-mesh abort with the proposed `MFEM_VERIFY` snippet guarding against `mesh.GetNodes()->FESpace()->GetMaxElementOrder() > 1`, and a note that curvilinear support requires per-QP Jacobian storage (deferred).
- Location: Phase 3 §"Edge Cases".

R-GPU-014 — Driver-owned ODE state needs **`UseDevice(true)`**

- What changed: (a) Phase 3 §"Files to Modify" gains a new bullet listing all four drivers and the contract: the ODE state vector must be flipped `UseDevice(true)` BEFORE `ode_solver->Init(...)`, with the code snippet to insert. (b) Phase 3 §"Edge Cases" gains the rationale with the ~3 TB PCIe traffic estimate. (c) Phase 3 acceptance criteria add a `< 5% wall-time in device transfers` check via `nvprof` / Nsight Systems.
- Location: Phase 3 §"Files to Modify" (drivers); §"Edge Cases"; §"Acceptance Criteria".

LOW (4/4)

R-GPU-015 — Makefile placeholder vagueness

- What changed: Phase 0 Makefile bullet rewritten to be specific:
  (a) read `MFEM_USE_CUDA` / `MFEM_USE_HIP` from `$(CONFIG_MK)`,
  (b) emit build-time errors on HYPRE-without-GPU mismatch,
  (c) declare an empty placeholder `SEAS_KERNEL_OBJS :=` variable that Phase 3 populates with `.cu` rules. Explicit deferral of nvcc / hipcc pattern rules to Phase 3.

- Location: Phase 0 §“Files to Modify”.

R-GPU-016 — PML kernel duplication risk between Phase 4 and Phase 5

- What changed: Phase 5 §“Files to Modify” `AdvanceADER` bullet rewritten to mandate a single `ApplyPMLDampingDeviceImpl(input, bg_scale, ...)` free function in `dynamic/kernels/pml_kernels.hpp` (added in Phase 4). Phase 4’s `ApplyPMLDamping` wraps with `bg_scale = 1.0`; Phase 5’s `ApplyPMLDampingADERDevice` wraps with `bg_scale = dt`. Explicit warning against separate copies.
- Location: Phase 5 §“Files to Modify”.

R-GPU-017 — GitHub Actions assumption

- What changed: Phase 7 `.github/workflows/seas_gpu_ci.yml` file replaced with `document/gpu_dev/CI_MATRIX.md` (documents the matrix in a CI-agnostic form). Plan now instructs the implementer to first check which CI system the repo uses (`ls .github/workflows/ .gitlab-ci.yml .buildkite/`) and then wire the matrix into the appropriate config.
- Location: Phase 7 §“Files to Modify” / new §“Files to Create (continued)”.

R-GPU-018 — “Avoid std::vector” wording too broad

- What changed: Convention constraints at top of plan reworded: the prohibition on `std::vector`, `std::map`, etc. is now scoped to INSIDE the kernel lambda body. Setup code on the host may use them freely.
- Location: §“Convention constraints”.

Unresolved Findings

None — all 18 findings are addressed in the plan document.

New Tests

None added as code (the artifact under repair is documentation). Test cases proposed in the review document remain as embedded acceptance criteria within each phase of the revised plan; they will be implemented when the corresponding phase is executed.

Deviations from Reviewer’s Suggested Fixes

None of substance. A few minor wording adjustments were made for flow (e.g., merging R-GPU-005 and R-GPU-018 into a single “Convention constraints” bullet block for cohesion). The technical content of every fix matches what the review specified.

Ready for Re-Review: YES

The plan is now consistent with the actual MFEM/HYPRE APIs verified during review, with explicit handling of: - Driver init order divergence (BP5 vs TPV) - Device-friendly virtual-dispatch alternative (enum + free fn) - Correct DOF counts and stack-array sizing for tets - Available atomic primitives only - Portable `mfem::AtomicAdd<T>` over raw CUDA / HIP atomics - HYPRE BoomerAMG GPU-vs-CPU tolerance realism - Explicit `Vector::Read()` / `Write()` capture in pseudocode - Distinct `Vector` vs `Array<T>` device-residency idioms - Batched (vdim=NUM_STATE) ghost-data path under MPI - Single-backend TOML validation - Independent face-table instances per owning class - Curvilinear-mesh abort - Driver-side `UseDevice(true)` flip on the ODE state - Explicit Makefile/CI gating - Shared PML kernel implementation - CI-agnostic build matrix documentation - Kernel-body-only restriction on `std::` containers